Multi-Task Heavy Ranking Network

Engineering architecture document · October 2026

MLflow Run ID	[paste from notebook output]
Azure ML registered model	`multi-task-ranking-model`
Azure ML model version	[paste from notebook output]
MLflow model URI (registration path)	`runs:/<run_id>/model`
MLflow experiment	multi_task_ranking_practice (on Databricks: /Users/<you>/multi_task_ranking_practice)

1. Why three objectives instead of click-through rate

The incumbent ranker optimises clicks alone, which rewards flashy content and lets long-term satisfaction slide. This model predicts three signals of deeper engagement in a single forward pass: whether the user will **Save** the item, how long they will **dwell** on it, and whether they will send it as a **Private Share**. The signals do not move together. Saves are frequent (32.5% of impressions in training), shares are rare (6.7%), and some content that drives saves suppresses sharing. A shared model trained naively can improve the frequent task at the expense of the rare, high-value one (negative transfer), so the design has to share what is common and keep what is task-specific separate, and the training run has to make interference visible.

2. Final model architecture

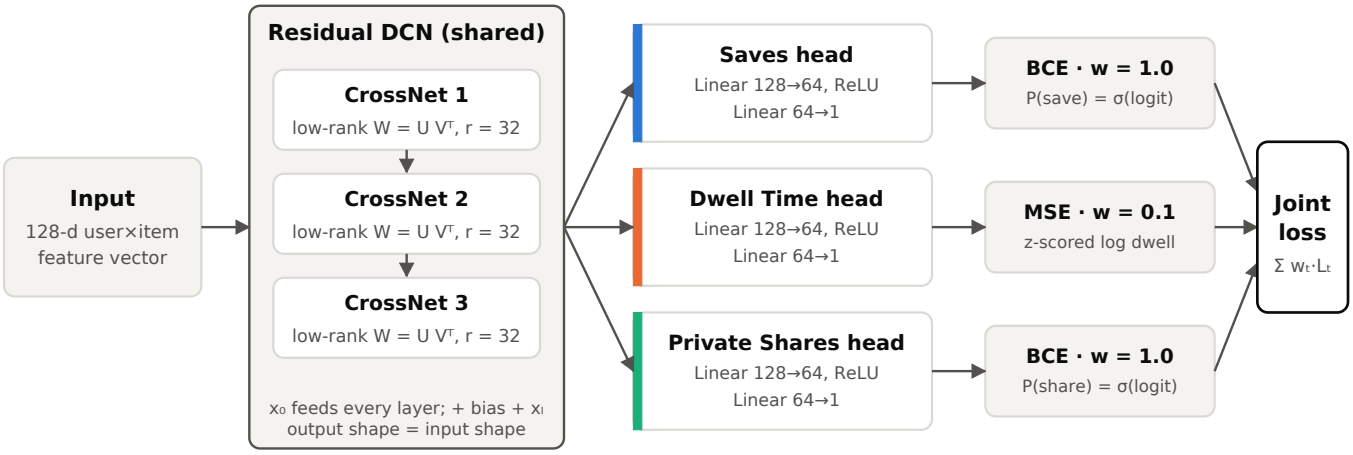


Figure 1. Shared Residual DCN trunk feeding three task towers. 49,923 trainable parameters in total; 24,960 of them in the cross layers.

- **Shared trunk: Residual DCN.** 3 stacked CrossNet layers compute $x_{l+1} = x_0 \odot (x_l\, V\, U^T) + b + x_l$. Each layer adds one degree of explicit feature interaction, so three layers model crosses up to fourth order without hand-built cross features. The original input x_0 is re-injected at every layer and the identity term keeps gradients flowing; the block preserves the input shape (128 → 128), which is what allows layers to be stacked.
- **Low-rank cross weights.** $W \approx U\, V^T$ with rank 32: x_l is projected down through V (128→32) and back up through U^T, costing 8,192 weights per layer instead of 16,384 for a full matrix (50% fewer at this width, and the saving grows with feature width).
- **Task towers.** Saves, Dwell Time and Private Shares each get their own Linear(128→64) → ReLU → Linear(64→1) head. The trunk learns interactions all three objectives use; the towers give each objective private capacity, which is the first line of defence against one task overwriting another.

- **Outputs.** Saves and Shares heads emit logits (σ gives probabilities at serving time); the Dwell Time head regresses z-scored log seconds, which tames the heavy right tail of raw dwell and puts its error on a scale comparable to the classification losses.

3. Joint objective and training

joint_loss = 1.0 · BCE(saves) + **0.1** · MSE(dwell) + 1.0 · BCE(shares). BCEWithLogitsLoss handles the two binary tasks; MSELoss handles dwell. MSE runs on a larger scale than BCE and, left unweighted, dominates the shared gradient, so it is down-weighted to 0.1. All weights are config fields, so rebalancing is a one-line change if validation AUC on Saves or Shares stalls while the joint loss keeps falling.

- AdamW (lr 0.001, weight decay 0.01), batch 512, gradient-norm clipping at 1; learning rate halves when validation joint loss stalls.
- Early stopping on validation combined AUC (patience 3); the best checkpoint is restored before logging. This run peaked at epoch 3 and stopped after epoch 6.
- Cross weights use Xavier initialisation. With the unit-variance torch.randn values in the starter code, a 4-epoch diagnostic on 60k rows produced DCN outputs with a standard deviation near 2.7×10^{5} at initialisation and all three AUCs stayed at about 0.50; Xavier reached 0.83 / 0.91 / 0.82 (Saves / Dwell / Shares) on the same budget.
- Data: 200,000 synthetic user×item impressions with 128 features, split 160,000 train / 40,000 validation with a fixed seed (42). Labels are driven by pairwise feature crosses, and one cross raises Saves while lowering Shares, so the trade-off described in section 1 is present in the data. Median dwell is 20.2 s; dwell is standardised with training-split statistics only.

4. Validation metrics

Metric	Value	Definition
Saves AUC	**0.8330**	ROC-AUC of σ(saves logit) against the Saved label
Dwell Time AUC	**0.9070**	ROC-AUC of predicted dwell against a long-dwell label (at or above the training median)
Private Shares AUC	**0.8783**	ROC-AUC of σ(shares logit) against the Shared label (6.7% positives)
Combined AUC	**0.8728**	Unweighted mean of the three per-task AUCs
Final joint loss (validation)	**0.6555**	1.0·BCE + 0.1·MSE + 1.0·BCE on the 40,000-row validation split
Final joint loss (training)	**0.6285**	Same objective, mean over the epoch 3 training batches
Dwell RMSE	**0.5074**	Root-mean-square error in z-scored log seconds

Table 1. Metrics for the registered checkpoint (epoch 3), evaluated on the held-out validation split. Combined AUC and validation joint loss are shaded as the headline figures.

Reading the curves for negative transfer. The run logs auc_saves, auc_dwell and auc_shares every epoch, plus a negative_transfer_flag that fires when Saves AUC rises while Shares AUC falls for 2 consecutive epochs. The flag never fired in this run: all three tasks peaked at epoch 3, and after that point the three moved together (Figure 2). The late decline is ordinary over-fitting (training loss keeps falling while validation loss rises for every task), which early stopping and checkpoint restore already handle. If a future run shows the scissor pattern instead (Saves up, Shares down), the remedies in order are: raise the Shares loss weight or add a positive-class weight, then move from a single shared trunk to gated experts (MMoE/PLE).

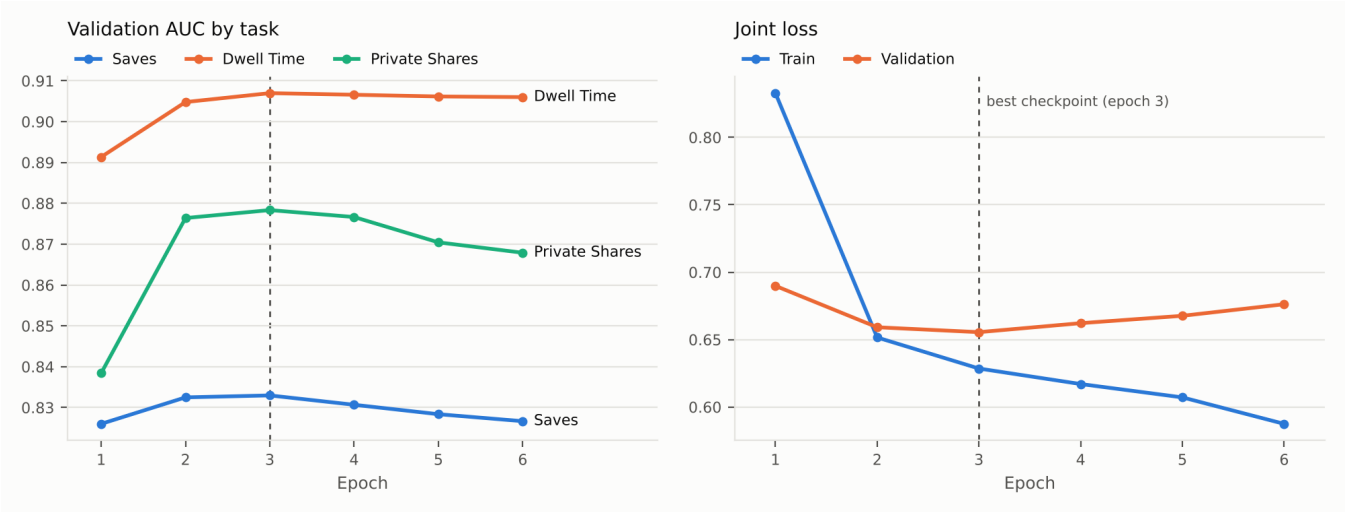


Figure 2. Per-task validation AUC (left) and joint loss (right) by epoch, as logged to MLflow. The dashed line marks the restored checkpoint.

5. Tracking and registration

- **MLflow.** Every hyper-parameter and data statistic is logged as a param. Each epoch logs joint_loss, val_joint_loss, the three per-task AUCs, combined_auc, per-task losses, dwell RMSE, the learning rate and the negative-transfer flag. The run also stores the training-curve figure, the epoch history as JSON, and the PyTorch model (mlflow.pytorch.log_model with an explicit signature and serialization_format="pickle").
- **Azure ML registry.** After the run, MLClient (DefaultAzureCredential) calls ml_client.models.create_or_update with Model(path=f"runs:/{run_id}/model", name="multi-task-ranking-model", type=AssetTypes.MLFLOW_MODEL). The run ID, model URI and per-task AUCs are attached as tags, and the assigned version is written back to the MLflow run as a tag, so registry entry and run point at each other.
- **Environment handling.** On Databricks the experiment uses the absolute /Users/<you>/… path Databricks requires. Elsewhere MLflow tracks into the Azure ML workspace when azureml-mlflow is installed, or locally otherwise. A runs:/ path only resolves inside the workspace that owns the run, so when it does not, the script downloads the logged model folder and registers that instead, keeping the run ID in the tags.

6. Before production

- Retrain on logged impressions with time-based splits; synthetic data validates the pipeline, not the business lift.
- Calibrate the Shares probability (Platt or isotonic) before it feeds a blended ranking score, and add per-segment AUC so rare-task regressions in small cohorts are not averaged away.
- Replace fixed loss weights with learned uncertainty weighting and A/B the blended score against the CTR model on saves, shares and 7-day return rate.